

Solution Code - Engineering concepts - Algorithms_DataStructures - Sorting_Customer_Orders

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: SortOrders.java

```
1 | class Order {
2 |     String id;
3 |     double totalPrice;
4 |     public Order(String id, double price) { this.id = id; this.totalPrice = price; }
5 | }
6 |
7 | public class SortOrders {
8 |     public static void bubbleSort(Order[] arr) {
9 |         int n = arr.length;
10 |         for (int i = 0; i < n-1; i++)
11 |             for (int j = 0; j < n-i-1; j++)
12 |                 if (arr[j].totalPrice > arr[j+1].totalPrice) {
13 |                     Order temp = arr[j];
14 |                     arr[j] = arr[j+1];
15 |                     arr[j+1] = temp;
16 |                 }
17 |     }
18 |
19 |     public static void quickSort(Order[] arr, int low, int high) {
20 |         if (low < high) {
21 |             int pi = partition(arr, low, high);
22 |             quickSort(arr, low, pi - 1);
23 |             quickSort(arr, pi + 1, high);
24 |         }
25 |     }
26 |
27 |     private static int partition(Order[] arr, int low, int high) {
28 |         double pivot = arr[high].totalPrice;
29 |         int i = (low - 1);
30 |         for (int j = low; j < high; j++) {
31 |             if (arr[j].totalPrice < pivot) {
32 |                 i++;
33 |                 Order temp = arr[i];
34 |                 arr[i] = arr[j];
35 |                 arr[j] = temp;
36 |             }
37 |         }
38 |         Order temp = arr[i+1];
39 |         arr[i+1] = arr[high];
40 |         arr[high] = temp;
41 |         return i + 1;
42 |     }
43 |     // Bubble sort is O(n^2), Quick sort is O(n log n) average.
```

44		}
----	--	---